Auteurs: Victor Chachereau, Anatole Liétar

Rapport de projet PIR - Partie Simulation

Sujet 15: Code Optimization and Simulation of a Self-Coordinating Robotic Aggregate

---

Python 3 research project focused on the dynamic simulation of a granulobot aggregate, modeled using a force-based approach to study the collective evolution of the system, contact interactions, magnetic effects, and the robots' rotational behavior.

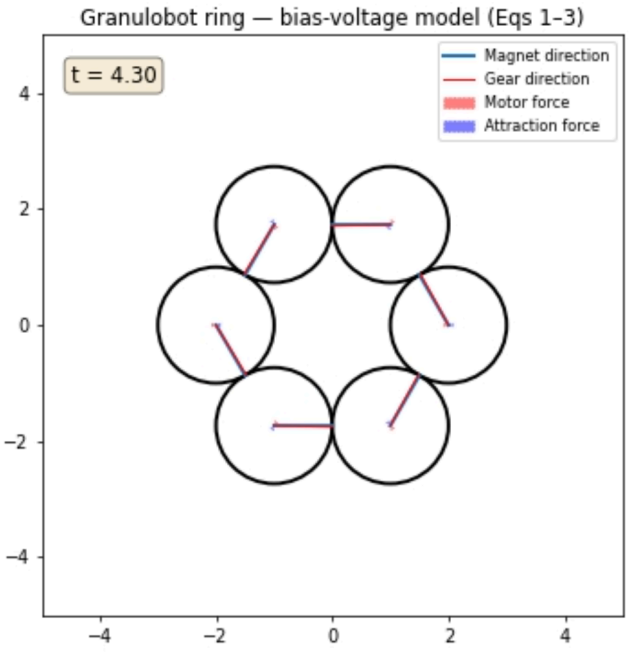


Table of contents:

Table of contents: **1**
Introduction **2**
Part I: Development and structuring of the simulator **2**
A. Optimization and improvement of the existing code (Anatole) 2
1.B.1 Quality of life improvements 2
1.A.2 Time optimisation with profiling tools 3
1.A.3 Trying newer rendering libraries 4
B. Verification, structuring, and parallelization of the simulator (Victor) 4
1.B.1 Reworking the existing code and identifying the modules 4
1.B.2 Multiprocessing parallelization and progress display 5
1.B.3 Contribution of this improvement to the project 6
Part II : Physical modeling and granulobot behavior **7**
A. Theoretical derivation of the ODE-based model (Anatole & Victor) 7
2.A.1 State variables of one granulobot 7
2.A.2 Numerical integration scheme 8
2.A.3 Translational dynamics 9
2.A.4 Neighbor link force 9
2.A.5 Neighbor link force 10
2.A.6 Coulomb friction clipping 11
2.A.7 Contact activation and overlap handling 11
2.A.8 Magnetic interaction 12
2.A.9 Motor torque and red/blue coupling 12
2.A.10 Rotational dynamics 13
2.A.11 Final ODE system 14
B. Qualitative analysis of physical realism and interaction calibration (Victor) 14
2.B.1 Objective of the force-based physical model 14
2.B.2 Identification of the three problems: blue line, red line, and overlap 15
2.B.3 Calibration of the constants and video comparison method 16
2.B.4 Results obtained and remaining limitations 17
C. Improvement of the dynamic model and qualitative validation of the simulated behaviors (Anatole) 17
Conclusion: **19**
Appendix **20**

Introduction

Granulobots are a promising area of particle and decentralized robotics in aggregate. While they achieve very interesting results, they are difficult to experiment with in practice. You have to charge the battery often, send the new code to the robots, establish and maintain a solid connection with all of them, have a reproducible initial state, repair the broken parts, etc. As such, testing new code and parameters to find interesting behaviours is time consuming.

This is where simulation comes in. With a faithful simulation of the granulobots, it should be possible to try out different parameters and initial states rapidly, and thus find working constants before testing them out in real life. But, a valid, in all scenarios, simulation tool for granulobots does not yet exist.

A big part of the physics implementation was already done by a previous student. Our work was to explore this area of research, by testing the existing codebase, validating each aspect of it, and improving the places that needed work.
It is basic to understand and simulate a single Granulobot, but an aggregate instantly turns a simple circle into a complex system with ever changing intricate inter-particules interactions, difficult and costly to simulate rightly.

Part I: Development and structuring of the simulator

A. Optimization and improvement of the existing code *(Anatole)*

1.B.1 Quality of life improvements

The very first part of our work consisted in reorganizing the existing codebase, because it consisted of a single python file with no instructions on how to use it or what libraries are needed to run the code.

Following conventions in the programming world, we moved the code to a dedicated git repo belonging to a Granulobot github organization that we also created. Having this enabled us to work simultaneously on the project white not affecting the other person, and after some progress had been made, to merge our modifications together. Also, we could try out new potentially code breaking features in separate branches. Finally, the git repo has a history of all changes committed to the code.

Then, the single file needed to be refactored into multiple files, each one dedicated to a part of the simulation program. Not only is the code clearer, it is also much simpler to work on a feature, conjointly with the git repo, while not affecting other people. To achieve this splitting, it was necessary to move the code to an object oriented approach, in which each class can have its own state that is kept in between steps, with a part of it being passed down to

another class. After this initial work, we had a clear base to start improving on the simulation.

We made small quality of life improvements throughout the project. Some of which are explained thoroughly in the following chapters, but here are a few examples. The command can be run with arguments to specify various aspects relative to its unfolding, like where the constant file is stored, where the results should be stored, in which mode to run the simulation, etc. The exported video includes a legend. The git repo includes a requirements file and a project file to have an easy to reproduce programming environment. There is now an explanation file : README.md, showing how to install the required tools, how to execute the code and the various arguments possible.

1.A.2 Time optimisation with profiling tools

The execution of the program is very long. To have an exploitable result in the form of a long video, it takes approximately 5 minutes. Trying out various constants and parameters suddenly becomes a tedious process. To solve this we investigated the code's run, in the hope of finding slow areas that could be sped up without changing the final result. We used profiling tools like pyinstruments to have a clear understanding of the execution time of each function.

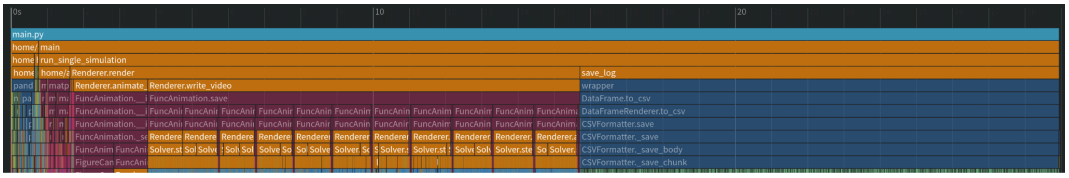


Figure 1 : Profiling results of the whole program. In orange at the bottom are each frame.

The figure 1 shows the whole execution of the program for 11 frames. As we can see, the *save_log* part at the right takes a bit less than half of the total time.

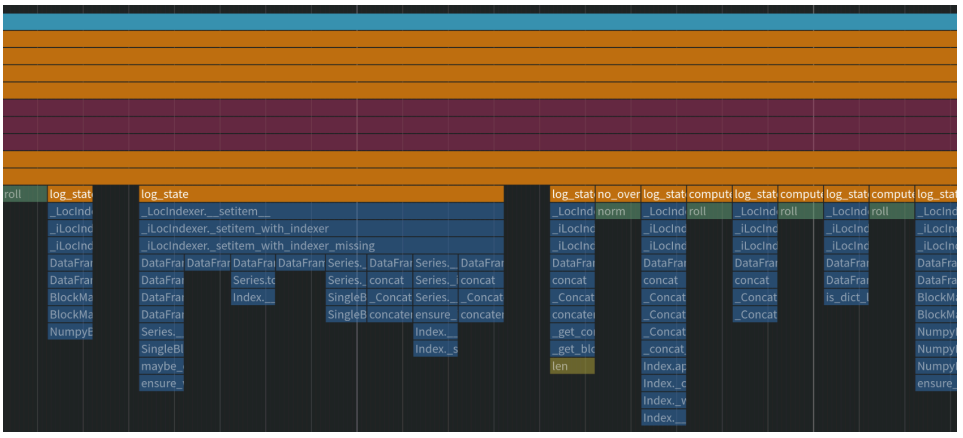


Figure 2 : Profiling results of a single step

When we enter into the details of one step, we also find a lot of time spent saving the logs. But, these logs are not often useful, especially when searching parameters rapidly, because the video results are enough to roughly validate the correctness of the simulation. As such,

we added a flag to the command's execution to run off the logs completely. This alone made the program run 50 to 80% faster while not affecting the end result.

1.A.3 Trying newer rendering libraries

Finally, the existing tool used the well known library matplotlib to render the simulation into a video. This library is very popular, versatile and easy to use, but is not particularly fast, especially when you need to generate hundreds of graphs to create a video. We did try two other graph rendering libraries (fastplotlib and pyqtgraph), the speed was increased but the video export was either too hard to implement or buggy so we abandoned this optimisation path. It could be interesting to explore more of this in the future to find out if a faster alternative is possible.

B. Verification, structuring, and parallelization of the simulator *(Victor)*

1.B.1 Reworking the existing code and identifying the modules

Before adding new features to the simulator, the first step was to review the existing code in order to understand its overall architecture and identify the role of each module. The simulator was already able to generate a video from a given set of constants, but it was necessary to precisely verify how the different parts of the program interacted with one another.

This verification phase made it possible to identify the complete execution chain of the program:

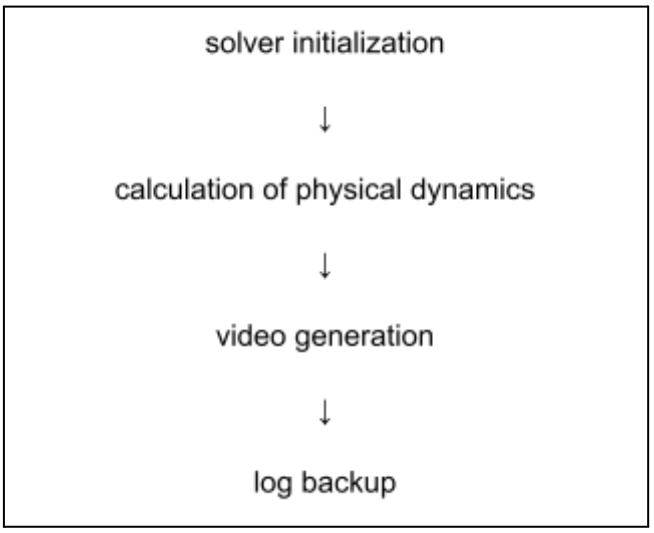


Figure 3 : initial code execution path

Special attention was paid to the physical elements already present in the code: interactions between neighboring granulobots, normal and tangential forces, friction effects, magnetic interaction, and motor torque. This step was important because the issues observed in the

videos could either result from a poor choice of constants or from a misinterpretation of the implemented model.

This initial verification therefore made it possible to clarify the responsibilities of each part of the code, while also highlighting several sensitive aspects of the simulator: numerical stability, contact handling, the apparent slipping of the blue and red lines, and the risk of overlap between granulobots.

1.B.2 Multiprocessing parallelization and progress display

Once the overall functioning of the simulator had been understood, an important improvement was made to the way simulations were executed. Initially, the program could only run one simulation at a time. However, a complete simulation could take a significant amount of time, mainly because of the frame-by-frame physical computation and the video encoding step.

This problem was well suited to parallelization, since each simulation is independent from the others. Indeed, a constants file corresponds to a complete experiment: it defines the physical parameters, produces a video, and may also generate a log file. Therefore, there is no dependency between two simulations using two different constants files.

The objective was therefore not to parallelize a single simulation, but rather to launch several simulations simultaneously. For this reason, a multiprocessing approach was chosen. Each process runs a complete simulation from a constants file. This makes it possible to use several CPU cores in parallel.

The principle can be summarized with the following pseudo-code:

```
configs = liste_des_fichiers_de_constantes

with ProcessPoolExecutor(max_workers=nombre_de_processus) as executor:
    for config in configs:
        executor.submit(run_single_simulation, config)
```

Figure 4: minimal pseudocode for the multiprocessing implementation

The `run_single_simulation()` function contains the logic of a complete simulation: loading the constants, initializing the simulation engine, generating the video, and saving the results. The role of the main program is then to distribute the different simulations across the available processes.

This approach provides a simple correspondence:

1 constants file
= 1 simulation

= 1 process
= 1 generated video

Thus, if four constants files are provided, as shown in the figure below, and four processes are allowed, four simulations can be executed simultaneously. This makes it possible to take advantage of several CPU cores, whereas a classical execution mainly used only one core.



Figure 5: On the left, a single simulation is launched: only one CPU core is used. On the right, several simulations are launched in parallel: several Python processes run simultaneously on multiple cores, making it possible to better exploit the available computing resources.

An additional difficulty came from the progress display. In the first version, each process wrote directly to the terminal, which made the messages unreadable when several simulations were running at the same time. To avoid this, the child processes now send their progress to the main process, which is the only one responsible for displaying it. This makes it possible to obtain a clean progress bar for each simulation.

1.B.3 Contribution of this improvement to the project

The addition of multiprocessing was directly useful for the rest of the project. Calibrating the simulator requires testing many sets of physical constants: contact stiffnesses, damping coefficients, friction values, magnetic interaction intensity, and coupling parameters between the blue and red lines. Without parallelization, these tests had to be launched one by one, which significantly slowed down the analysis.

Thanks to multiprocessing, several configurations can be evaluated in parallel. The simulator therefore becomes a more efficient experimentation tool: it is possible to launch several videos corresponding to different choices of constants, and then qualitatively compare the resulting behaviors.

This improvement notably helped speed up the physical model tuning phases. Tests related to the slipping of the blue line, the behavior of the red line, and the overlaps between granulobots could be carried out more quickly, because several hypotheses could be tested simultaneously.

In practice, this contribution does not directly modify the physical model, but it greatly improves the ability to explore it. It transforms the simulator into a tool that is better suited to

a research approach: instead of testing parameters sequentially, it becomes possible to quickly compare several scenarios and more efficiently identify the most promising configurations.

In summary, multiprocessing made it possible to move from a simulator capable of producing only one video at a time to a tool capable of launching several experiments in parallel, which represented an important gain for studying and calibrating the collective behavior of the granulobots.

Part II : Physical modeling and granulobot behavior

A. Theoretical derivation of the ODE-based model (Anatole & Victor)

Before calibrating the physical parameters of the simulator, it is necessary to describe the theoretical model on which the code is based. The simulator does not impose exact geometric constraints using a Lagrangian formulation or Lagrange multipliers. Instead, it uses a force-based approach: the expected behavior of the aggregate is approximated through interaction forces, damping terms, friction, magnetic effects, and motor torques.

The objective of this section is to explain how the dynamics of the granulobot aggregate can be written as a system of ordinary differential equations (ODEs), and how the expressions used in the simulator are obtained.

2.A.1 State variables of one granulobot

Each granulobot is represented by a disk of radius (R), where (R) corresponds to gear_radius in the code. For each granulobot (i), the translational variables are:

$$\mathbf{r}_i = \begin{pmatrix} x_i & y_i \end{pmatrix}$$

the position of the center of the robot, and

$$\boldsymbol{v}_i = \dot{\boldsymbol{r}}_i$$

the velocity.

The rotational state is described by two internal angles:

$$\theta_i$$

which represents the gear orientation, and

$$\phi_i$$

which represents the magnet orientation. Their corresponding angular velocities are:

$$\omega_{\theta,i} = \dot{\theta}_i$$

and

$$\omega_{\phi,i} = \dot{\phi}_i$$

Therefore, the state of one granulobot can be written as:

$$Y_i = (\boldsymbol{r}_i, \boldsymbol{v}_i, \theta_i, \omega_{\theta,i}, \phi_i, \omega_{\phi,i})$$

For an aggregate of (N) granulobots, the full state vector is:

$$Y = (Y_1, Y_2, \ldots, Y_N)$$

The goal of the simulator is to compute the time evolution of this state vector, which can be written in the general ODE form:

$$\dot{Y} = f(Y)$$

At each time step, the simulator computes all forces and torques from the current state, and then updates the positions, angles, velocities, and angular velocities.

2.A.2 Numerical integration scheme

The simulation advances with a time step (dt). At each step, the positions and angles are first updated using the current velocities:

$$\mathbf{r}_i(t + dt) = \mathbf{r}_i(t) + \mathbf{v}_i(t)dt$$

$$\theta_i(t + dt) = \theta_i(t) + \omega_{\theta,i}(t)dt$$

$$\phi_i(t + dt) = \phi_i(t) + \omega_{\phi,i}(t)dt$$

Then the simulator computes the interaction forces and torques, and updates the velocities:

$$\mathbf{v}_i(t + dt) = \mathbf{v}_i(t) + \mathbf{a}_i(t)dt$$

$$\omega_{\theta,i}(t+dt) = \omega_{\theta,i}(t) + \dot{\omega}_{\theta,i}(t)dt$$

$$\omega_{\phi,i}(t+dt) = \omega_{\phi,i}(t) + \dot{\omega}_{\phi,i}(t)dt$$

This gives a discrete approximation of the continuous ODE system.

2.A.3 Translational dynamics

The translational motion of each granulobot follows Newton's second law:

$$m_i \ddot{\mathbf{r}}_i = \mathbf{F}_i$$

where m_i is the mass of granulobot i, and $\boldsymbol{F}_i$ is the total force applied to it. In the simulator, this total force is the sum of three main contributions:

$$\boldsymbol{F}_i = \boldsymbol{F}_i^{gear} + \boldsymbol{F}_i^{mag} + \boldsymbol{F}_i^{link}$$

where:

- $\mathbf{F}_i^{gear}$ is the force generated by gear/contact interactions,
- $\mathbf{F}_i^{mag}$ is the magnetic interaction force,
- $\mathbf{F}_i^{link}$ is a stabilizing force that keeps neighboring granulobots close to their expected distance.

Therefore, the translational part of the ODE is:

$$\dot{\mathbf{r}}_i = \mathbf{v}_i$$

$$\dot{\mathbf{v}}_i = \frac{\mathbf{F}_i^{gear} + \mathbf{F}_i^{mag} + \mathbf{F}_i^{link}}{m_i}$$

2.A.4 Neighbor link force

The initial configuration is an aggregate arranged as a ring. To help preserve this structure, the simulator uses a spring-damper link between direct neighbors. The target distance between two neighboring granulobots is:

$$d_0 = 2R$$

For two neighboring robots i and j=i+1, we define:

$$\mathbf{d}_{ij} = \mathbf{r}_j - \mathbf{r}_i$$

$$L_{ij} = ||\mathbf{d}_{ij}||$$

$$\mathbf{n}ij = \frac{\mathbf{d}ij}{L_{ij}}$$

The distance error is:

$$e_{ij} = L_{ij} - d_0$$

and the relative normal velocity is:

$$v_{rel,ij} = (\mathbf{v}_j - \mathbf{v}i) \cdot \mathbf{n}ij$$

The stabilizing link force is then:

$$\boldsymbol{F}_{ij}^{link} = (-k_{link} e_{ij} - c_{link} v_{rel,ij}) \boldsymbol{n}_{ij}$$

This term pulls neighboring granulobots back toward the target distance d_0, while the damping term prevents excessive oscillations.

2.A.5 Gear/contact interaction

For each pair of granulobots, the simulator computes the relative displacement:

$$\boldsymbol{\delta}_{ij} = \mathbf{r}_j - \mathbf{r}_i$$

$$d_{ij} = ||\boldsymbol{\delta}_{ij}||$$

The unit normal vector is:

$$\boldsymbol{e}_{ij} = \frac{[\backslash \text{boldsymbol?}]\delta_{ij}}{d_{ij}}$$

and the associated tangential direction is:

$$\boldsymbol{t}_{ij} = (-e_{ij,y}, e_{ij,x})$$

The relative velocity is:

$$\mathbf{v}_{rel,ij} = \mathbf{v}_j - \mathbf{v}_i$$

The normal relative velocity is:

$$d_{norm,ij} = \boldsymbol{v}_{rel,ij} \cdot \boldsymbol{e}_{ij}$$

The tangential slip term is:

$$d_{tan,ij} = \boldsymbol{v}_{rel,ij} \cdot \boldsymbol{t}_{ij} - \frac{d_{ij}}{2}(\omega_{\theta,i} + \omega_{\theta,j})$$

This expression combines tangential translation and rotational slip. It is used to model the tendency of two contacting granulobots to slide against each other.

The tangential and normal interaction terms are then computed as:

$$F_{tan,ij} = k_{tan} d_{tan,ij}$$

$$F_{norm,ij} = k_{norm} d_{norm,ij}$$

Damping terms are added:

$$F^{tot}_{tan,ij} = F_{tan,ij} + \zeta_t d_{tan,ij}$$

$$F^{tot}_{norm,ij} = F_{norm,ij} + \zeta_n d_{norm,ij}$$

The resulting gear/contact force is:

$$\boldsymbol{F}^{gear}_{ij} = F^{tot}_{tan,ij}\boldsymbol{t}_{ij} + F^{tot}_{norm,ij}\boldsymbol{e}_{ij}$$

For each granulobot, the net gear force is obtained by summing all pairwise contributions:

$$\boldsymbol{F}^{gear}_{i} = \sum_j \boldsymbol{F}^{gear}_{ij}$$

The gear/contact torque used by the simulator is:

$$T^{gear}_{ij} = F^{tot}_{tan,ij}(F_{norm,ij} + d_0)$$

and the net gear torque is:

$$T^{gear}_{i} = \sum_j T^{gear}_{ij}$$

2.A.6 Coulomb friction clipping

To avoid unrealistic tangential forces, the simulator applies a Coulomb friction limitation. The tangential force is clipped so that:

$$|F_{tan,ij}| \leq \mu |F_{norm,ij}|$$

This means that the tangential force cannot grow indefinitely. The coefficient μ controls the maximum friction level between granulobots.

2.A.7 Contact activation and overlap handling

The model does not solve exact non-overlap constraints. Instead, it activates or deactivates contact forces depending on the distance between robots. If two granulobots are farther apart than the reference distance d_0, their gear/contact force is set to zero:

$$\|\delta_{ij}\| \geq d_0 \Rightarrow \boldsymbol{F}^{gear}_{ij} = 0$$

This avoids applying contact forces between robots that are not close enough to interact. However, direct neighbors in the ring are kept active in order to preserve the aggregate structure.

This explains why overlap control depends strongly on the balance between the contact constants, the stabilizing link force, and the integration time step (dt). If the constants are too

weak, the robots may overlap. If they are too strong, the numerical integration may become unstable.

2.A.8 Magnetic interaction

The magnetic direction of each robot is described by the internal angle ϕ_i. In the simulator, a fixed preset angle is also added to account for the initial geometry of the ring. The magnetic point associated with robot (i) is modeled as:

$$\boldsymbol{m}_i = 2R\begin{pmatrix}\cos(\phi_i+\phi_{preset,i})\\ \sin(\phi_i+\phi_{preset,i})\end{pmatrix}$$

The tangent direction associated with this magnetic point is:

$$\mathbf{t}_i^{mag} = R\begin{pmatrix}-\sin(\phi_i+\phi_{preset,i})\\ \cos(\phi_i+\phi_{preset,i})\end{pmatrix}$$

The magnetic displacement is:

$$\boldsymbol{d}_i^{mag} = \delta_{i,1} - \boldsymbol{m}_i$$

The forward magnetic force is:

$$\boldsymbol{F}_i^{forward} = k_m \boldsymbol{d}_i^{mag} - \zeta_m \omega_{\phi,i} \boldsymbol{t}_i^{mag}$$

The first term acts as a magnetic restoring force. The second term is a damping term that limits unstable oscillations of the magnetic orientation.

The net magnetic force applied to robot i is computed as:

$$\boldsymbol{F}_i^{mag} = \boldsymbol{F}_i^{forward} - \boldsymbol{F}_{i-1}^{forward}$$

The corresponding magnetic torque is:

$$T_i^{mag} = \boldsymbol{F}_i^{forward} \cdot \boldsymbol{t}_i^{mag}$$

2.A.9 Motor torque and red/blue coupling

The simulator also includes an internal motor model that couples the two angles (\theta_i) and (\phi_i). The internal control variable is:

$$U_i = \mathrm{sgn}(\theta_i - \phi_i)u_i - \frac{\alpha\eta_0}{k_i}(\omega_{\theta,i} - \omega_{\phi,i}) - \frac{G}{k_i}(\theta_i - \phi_i)$$

The corresponding drive torque is:

$$T_i^{drive} = k_i U_i$$

or, equivalently:

$$T_i^{drive} = k_i \operatorname{sgn}(\theta_i - \phi_i) u_i - \alpha \eta_0 (\omega_{\theta,i} - \omega_{\phi,i}) - G(\theta_i - \phi_i)$$

This expression contains three effects:

- a motor command term controlled by u_i,
- a damping term depending on the relative angular velocity,
- a torsional restoring term depending on the angular difference $\theta_i - \phi_i$

The parameter G therefore controls how strongly the gear and magnet orientations are coupled. A larger value tends to reduce the angular difference between the two internal directions, while a smaller value allows more independent motion.

The simulator then defines the load torque:

$$\Gamma_{l,i} = T_i^{gear} - T_i^{mag}$$

and a resistant torque:

$$T_i^{res} = T_i^{drive} - \Gamma_f \operatorname{sgn}(\omega_{\theta,i} - \omega_{\phi,i}) - \eta_0(\omega_{\theta,i} - \omega_{\phi,i})$$

The torque adjustment is computed as:

$$T_i^{adjust} = k_{eq} \begin{cases} \eta_0(\omega_{\phi,i} - \omega_{\theta,i}) & \text{if } |\Gamma_{l,i} + T_i^{drive}| \leq \Gamma_f \\ \Gamma_{l,i} + T_i^{res} & \text{otherwise} \end{cases} - \Gamma_{l,i}$$

The final torques applied to the two angular variables are:

$$T_{\theta,i} = T_i^{gear} + \frac{T_i^{adjust}}{2}$$

$$T_{\phi,i} = T_i^{mag} - \frac{T_i^{adjust}}{2}$$

This coupling is central for reproducing the behavior of the two internal orientations visible in the simulation.

2.A.10 Rotational dynamics

The rotational dynamics are written as:

$$\dot{\theta}_i = \omega_{\theta,i}$$

$$\dot{\phi}_i = \omega_{\phi,i}$$

The angular accelerations are obtained from the torques. In the current implementation, the effective inertial terms are distributed using the parameter rot_rat, (annotated ρ):

$$I_{\theta,i}^{eff} = (1-\rho)m_i$$

$$I_{\phi,i}^{eff} = \rho m_i$$

Therefore:

$$\dot{\omega}_{\theta,i} = \frac{T_{\theta,i}}{(1-\rho)m_i}$$

$$\dot{\omega}_{\phi,i} = \frac{T_{\phi,i}}{\rho m_i}$$

2.A.11 Final ODE system

Combining the previous equations, the dynamics of each granulobot can be summarized as a system of coupled ordinary differential equations. For each granulobot i, the translational dynamics are:

$$\dot{\mathbf{r}}_i = \mathbf{v}_i$$

$$\dot{\mathbf{v}}_i = \frac{\mathbf{F}_i^{gear} + \mathbf{F}_i^{mag} + \mathbf{F}_i^{link}}{m_i}$$

The rotational dynamics are:

$$\dot{\theta}_i = \omega_{\theta,i}$$

$$\dot{\phi}_i = \omega_{\phi,i}$$

Using $\rho = rot_rat$, the angular accelerations are:

$$\dot{\omega}_{\theta,i} = \frac{T_{\theta,i}}{(1-\rho)m_i}$$

$$\dot{\omega}_{\phi,i} = \frac{T_{\phi,i}}{\rho m_i}$$

where $T_{\theta,i}$ and $T_{\phi,i}$ are the final torques applied respectively to the gear and magnet orientations, as defined in the previous section.

This system of coupled ODEs is integrated numerically over time. Since all granulobots interact with one another through contact, friction, magnetic effects, stabilizing link forces, and internal motor coupling, small changes in the physical constants can strongly affect the resulting behavior. This explains why the calibration phase is essential: the values of k_{tan}, k_{norm}, k_m, ζ_t, ζ_n, ζ_m, μ, G, k_{eq}, and dt directly influence stability, slipping, synchronization, and overlap.

B. Qualitative analysis of physical realism and interaction calibration (Victor)

2.B.1 Objective of the force-based physical model

Based on the ODE model derived above, the next step was to evaluate whether the simulated dynamics produced physically credible behaviors. The model used in the simulator does not rely on an exact resolution of geometric constraints, as could be done with a Lagrangian formulation and Lagrange multipliers. Instead, it follows a force-based approach, where the expected behaviors are approximated through contact forces, friction, damping, restoring forces, magnetic interactions, and motor torques. The objective was therefore not only to obtain a global motion of the aggregate, but also to verify that the internal orientations, contact interactions, and collective motion remained consistent with the expected granulobot behavior.

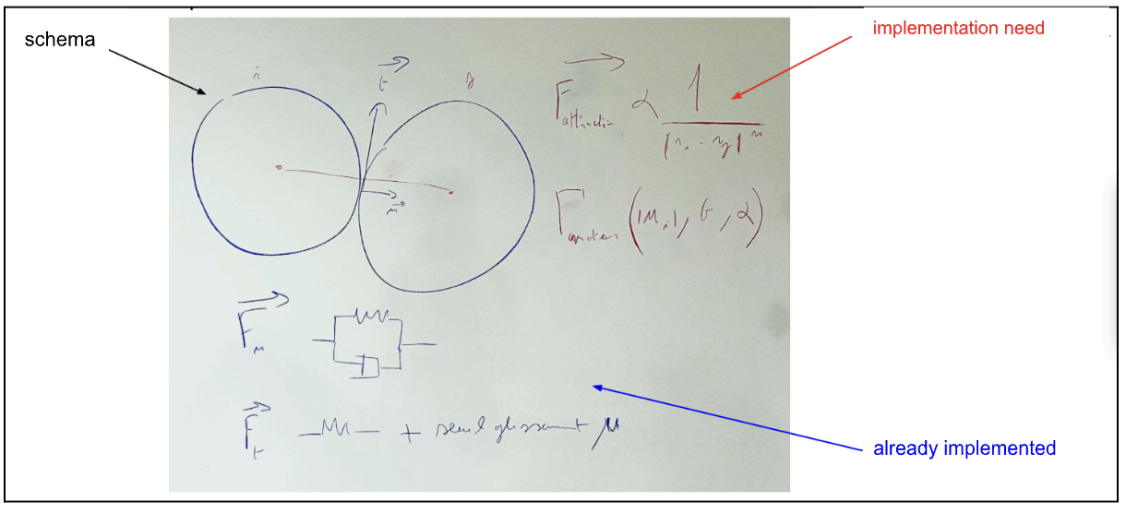


Figure 6: Illustration of the force-based approach

In this simulation, each granulobot is represented by a disk, with two colored lines associated with it. The blue line and the red line are used to visualize the robot's internal orientations. These orientations are important because they reflect the behavior of the motor/magnet system and its synchronization with the contacts between neighboring granulobots.

The objective was therefore not only to obtain a global motion of the aggregate, but also to achieve a qualitatively credible behavior. To do so, several criteria were observed in the generated videos: the stability of the aggregate, the consistency of the blue and red lines, the absence of significant overlap between robots, and a motion that was dynamic enough without becoming unstable.

2.B.2 Identification of the three problems: blue line, red line, and overlap

The first simulations highlighted several realism issues. Three main points were identified: the behavior of the blue line, the behavior of the red line, and the overlaps between granulobots.

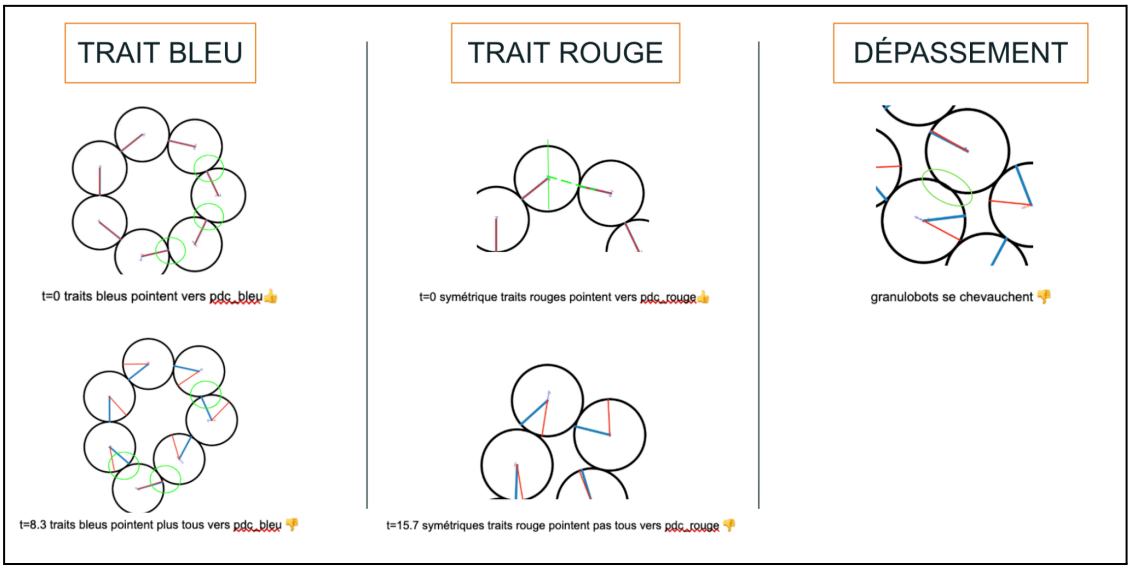


Figure 7: Summary table of realism issues

The first problem concerned the blue line : at initialization, the blue lines correctly pointed toward the expected contact points. However, during the simulation, they tended to gradually desynchronize. This reflected a slipping effect or a delay in the model, which prevented good consistency from being maintained between the visualized orientation and the actual contact between the robots.

The second problem concerned the red line : it did not necessarily have to remain perfectly aligned with the blue line. Based on discussions with the supervisor, the expected behavior was rather an oscillation of the red line around the blue line. The initial problem was that the red lines could sometimes behave too freely or be insufficiently correlated, which made the robot's internal dynamics less credible.

The third problem concerned overlap : in other words, the overlapping of granulobots. Since the geometric constraints are not imposed exactly, two robots may overlap if the normal contact forces are not strong enough or if the time step is too large. This problem was particularly important because it directly challenged the visual validity of the simulation.

2.B.3 Calibration of the constants and video comparison method

To correct these problems, the approach consisted in progressively varying the physical constants of the model and qualitatively analyzing the videos obtained. This calibration was carried out in a pseudo-iterative way: several configurations were tested, the videos were observed, and then the parameters were modified according to the visible defects.

```
# USER CONTROLS
n: 7
gear_radius: 1

gear_params:
  - {bias_u: 0.75, k: 0.229, m: 2.0}
  - {bias_u: 0.75, k: 0.229, m: 2.0}
  - {bias_u: 0.75, k: 0.229, m: 2.0}
  - {bias_u: 0.75, k: 0.229, m: 2.0}
  - {bias_u: 0.75, k: 0.229, m: 2.0}
  - {bias_u: 0.75, k: 0.229, m: 2.0}
  - {bias_u: 0.75, k: 0.229, m: 2.0}

# Motor / drive constants
eta0: 0.65
Gamma_f: 0.024
alpha: -1

# Red/blue coupling
G: 1.2
I_theta: 0.015
I_phi: 0.001
rot_rat: 0.5

scale: 0.0
damp_coeff: 50

# Contact / interaction parameters
k_tan: 145.0
k_norm: 350.0
k_m: 28.0
k_eq: 0.45

zeta_n: 550.0
zeta_t: 300.0
zeta_m: 800.0

mu: 30.0

# Integration
dt: 0.0002
steps_per_frame: 500
frames: 900
fps: 30

neighbor_iters: 6
global_iters: 2
w_clip: 12.0
```

Figure 8 : Example of a constants file (.yaml)

For the blue line, the parameters mainly studied were those related to magnetic interaction and tangential friction. Increasing `k_m` strengthened the magnetic effect, while `zeta_m` damped this interaction in order to avoid the system becoming unstable. The parameters `k_tan`, `zeta_t`, and `mu` were also adjusted to limit tangential slipping between granulobots.

For the red line, the most important parameters were related to the coupling between the two internal orientations of the robot. In particular, `G` acts as a torsional restoring term between the internal angles, while `k_eq` modulates the dynamic balancing of torques. The objective was not to completely lock the red line onto the blue line, but to obtain a correlated dynamic behavior, with reasonable oscillations around the blue orientation.

Finally, to limit overlaps, the normal contact parameters were adjusted. Increasing `k_norm` makes the contact more rigid, while `zeta_n` damps excessively fast approaches. The time step `dt` also played an important role: when the constants become too stiff, a time step that is too large can lead to numerical instabilities, with infinite or undefined values appearing in the simulation.

The main difficulty was to find a compromise. Constants that were too low led to visible slipping or contacts that were too soft. Conversely, constants that were too high made the simulation unstable, with overly abrupt movements or numerical errors. The work therefore consisted in progressively approaching an acceptable regime without making the system diverge.

2.B.4 Results obtained and remaining limitations

The latest simulations showed a clear improvement compared to the first versions. The overall behavior of the aggregate was more stable, significant overlaps were greatly reduced, and the blue and red lines showed better consistency with the expected motion. At this stage:

The behavior of the blue line remains perfectible. It is now much closer to the contact point than at the beginning, but a slight slipping effect still remains after some time in the simulation. This shows that synchronization has not been fully achieved yet, although the model is getting closer to it.

The behavior of the red line is also more satisfactory. Instead of moving too independently, it now appears more correlated with the blue line. Its behavior better matches the idea of an oscillation around the blue orientation, even though this tuning remains sensitive to the chosen constants.

Regarding overlaps, the improvements are clearer. Significant overlaps between granulobots have become much rarer and now mainly fall under fine-tuning rather than being a major flaw of the model. The latest adjustments therefore mostly focused on line synchronization and on the balance between system energy and numerical stability.

C. Improvement of the dynamic model and qualitative validation of the simulated behaviors (Anatole)

The ultimate goal of establishing a stable and fast simulator is to easily discover the best combinations of variables without having to guess them manually. Since the parameter space is vast, we implemented an automated pipeline designed to systematically test a wide range of values, analyze their outcomes through physical metrics, and score them to isolate the most effective configurations.

The first step of this process involved modifying the configuration system to support variable ranges instead of static values. Instead of running a single simulation with one rigid set of constants, the program can now ingest a predefined interval for specific parameters. The simulator automatically iterates through these ranges, spawning simulations running in parallel. While the resulting videos are easy to analyze and conclude on to choose the good parameters within the range, we thought of an automatic way of selecting the best run.

To evaluate the success of each run, the second step requires tracking and logging the swarm's physical variables over time. We implemented tracking for several key metrics:

- **Planar movement and velocity:** To assess how effectively the aggregate translates across space.
- **Swarm area:** A crucial metric to measure the expansion and contraction of the aggregate, which is directly tied to the "breathing" behavior of the robot cluster.
- **Phase shift between Granulobots:** Essential for analyzing how well individual units synchronize their internal cycles with one another.

Once these physical variables are extracted, the third step consists of defining a single indicator to quantify the overall "quality" of the simulation. For synchronization behaviors, this quality score is calculated using the mean of the phase shifts across all particles. A lower average phase shift indicates a high degree of collective synchronization, serving as a clear benchmark for success.

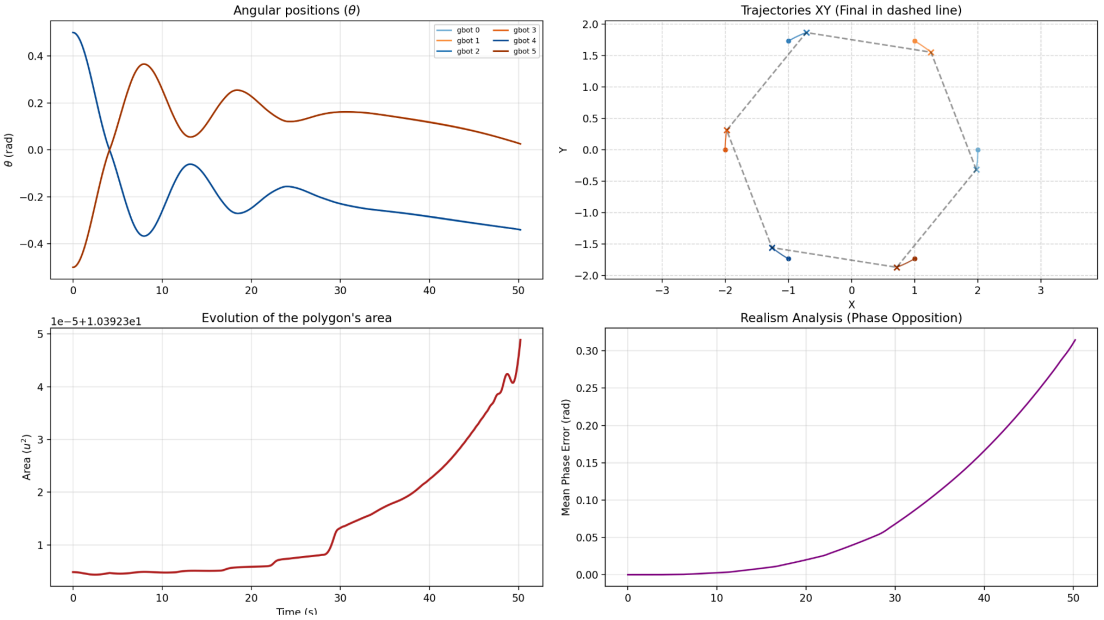


Figure 9: metrics quantifying the simulation's veracity. In blue tone are even robots and in red tone are odd robots.

The angular position graph tells us that after two oscillations the robots are completely desynchronized. This behaviour is confirmed by the evolution of the polygon's area graph. It shows how the aggregate starts to breathe while it should stay stable in theory. The corresponding video is: https://drive.google.com/file/d/1sAYhLuYQ3HV_GZJmcDiutWUvvJXrZ51R/view?usp=drive_link. We can easily conclude that the used parameters are not yet satisfying and still need more trial and error.

Moving forward, the next phase of development will focus on closing the loop of this optimization process. The program will automatically select the best-performing simulation within the tested range based on its quality score. From there, it will dynamically generate and test a new set of surrounding values, iteratively narrowing down on the absolute best parameters to maximize the synchronization score.

Conclusion:

The main goal of this project was to transform a single, unstructured Python script into a reliable and efficient simulator for granulobot aggregates. Simulating a decentralized swarm is tough because the constant interactions between particles make it hard to balance physical realism with computing speed. By combining code optimization, physical tuning, and automation, we managed to build a solid tool for future research.

First, we focused on cleaning up and speeding up the program. Moving the code to an object-oriented structure and setting up a clean GitHub repository made working together much easier. By profiling the code, we cut out unnecessary logging and made the program 50% to 80% faster. We also added multiprocessing, meaning we can now run several simulations at the same time across multiple CPU cores instead of waiting for them one by one.

This extra speed was crucial for the second part of our work: refining the physical model. By testing different constants iteratively, we fixed major issues like robots overlapping and adjusted the behaviors of the internal blue and red lines. While fine-tuning the exact synchronization and tangential slipping still needs some work, the overall aggregate behavior is now much more stable and realistic.

Lastly, we started moving away from manual trial-and-error by designing an automated optimization pipeline. The simulator can now run a range of parameters, track key physical metrics (like velocity, swarm area, and phase shifts), and calculate a quality score based on how well the robots synchronize.

We haven't built a perfect simulator for every single real-world scenario yet, but we have turned a slow script into a real experimentation tool. The next step will be to fully close the automation loop so the code can find the best parameters on its own, and eventually test these results against real physical granulobots.

Appendix

Link to the code :
https://drive.google.com/drive/folders/1IwOuEOFEglOjtnz79DD0KLrB8zAPVfoY?usp=drive_link

Link to the videos :
https://drive.google.com/drive/folders/1S5lYZyB8FJcPLe9GMYsxXxGFUk_ReVzL?usp=drive_link